Merge-Based Vocabulary Extension Stalls at the Pre-Tokenizer: An English-Preserving Repair for Nepali, Tested in Continued Pretraining

Aarjan Chaudhary
Naamche Labs

Abstract. **Llama-3, Qwen3 and many other open LLMs pre-tokenize text with a regular expression whose word class admits only Unicode letters. Devanagari vowel signs are combining marks, so this regex splits Nepali words at every vowel sign, and models that use it spend 2.6 to 5.8 times as many tokens on Nepali as on the same English text. Merge-based vocabulary extension cannot join the split pieces: for Llama-3 and Qwen3 no number of new merges brings this premium below about 2.2, and ten released extensions already end within 4% of this pre-token floor. We give a small edit to the deployed regex that provably keeps every token id of text without combining marks or Devanagari characters. With 32,000 new merges it brings Nepali to 1.01 times English, about 54% fewer Nepali tokens than standard extension. In small-budget continued pretraining of Llama-3.2-1B and Qwen3-0.6B, the original tokenizer gave the lowest held-out Nepali bits per byte under every recipe we tested. Matching the repair's training steps to standard extension, with about 35% of the data repeated, reversed their order on Qwen but not on Llama. Whether the token savings give a favourable latency and quality trade-off in generation remains untested. We release the tokenizers and code.**

I. Introduction

On parallel FLORES-200 text, Qwen3's tokenizer needs 4.42 times as many tokens for a Nepali sentence as for its English translation, and the `cl100k` tokenizer used by Phi-4, OLMo-2 and Granite-4.1 needs 4.76 times as many (§IV). We call this ratio the *premium*. Nepali speakers pay it on every request, in price, latency and usable context.

Much of the premium has a mechanical cause. GPT-2-style pre-tokenizers split text with a regular expression whose word class `\p{L}` matches one or more Unicode letters [1]; we call it the *letters-only regex*. In Unicode, Devanagari vowel signs, the virama, the anusvara and the candrabindu are combining marks (`\p{M}`), so the regex starts a new pre-token at each of them and byte-pair encoding (BPE) can never merge across that boundary. Regmi et al. [2] formalised this cause and the resulting bound on token counts. They showed that adding marks to the word class cuts Nepali from 4.78 to 1.58 tokens per word, and that 63% of the 1,345 text-generation repositories they classified still use the letters-only class.

Regmi et al. studied tokenizers trained from scratch. Most practitioners instead adapt a model that already exists, usually by vocabulary extension: they add target-language tokens, initialise their embeddings and continue pretraining [3–6]. We study that deployed-model setting. We show that the bound becomes a floor that released merge-based extensions already hit; we give a repair of a deployed model's regex that provably keeps English token ids; and we test it in controlled continued pretraining against the standard continued-BPE recipe of Purason et al. [5], with the tokenizers released.

Findings. For Llama-3 and Qwen3 the *pre-token floor*, the lowest premium any merge-based extension can reach, is about 2.2 (§III). Ten released extensions for six scripts end within 4% of their floors, and our own extension under the letters-only regex stalls at it. Registering our repair's learned strings as added tokens [cf. 7] gets below the floor, but a fifth of the matches then start mid-word. Our repair adds marks to the word class with two string substitutions and, with new merges, brings Nepali to parity with English while leaving every token id of mark-free, non-Devanagari text unchanged (§V). The model-quality result is less favourable. Continued pretraining with no new tokens gave the best held-out Nepali bits per byte on both models, so our main hypothesis failed. At equal bytes the repair is worse than standard extension (0.025 bits per byte on Llama and 0.012 on Qwen, two-seed means), but it takes 26% fewer steps; at matched steps it is 0.010 better on Qwen and 0.012 worse on Llama, and a higher learning rate on Qwen improves all three arms' Nepali scores without changing their order, at a cost on English (§VII-B). For the same Nepali text the repair needs 54% fewer tokens than standard extension; whether this speeds up generation of equally good text is untested.

Contributions. We make four contributions:

1) Evidence that the pre-token floor binds released merge-based extensions, and a test of the added-token route around it (§III); an audit of 24 distinct deployed tokenizers with per-tokenizer floors (§IV).
2) A repair of a deployed regex that provably keeps English token ids (not model behaviour), a Devanagari-only variant with a stronger guarantee (not trained), and a practical pitfall in loading repaired tokenizers (§V).
3) A controlled comparison of R0 (no extension), R1 (standard extension) and R2 (repair + extension) in continued pretraining of two deployed models, reported against hypotheses written down in advance, including the one that failed (§VII-B).
4) The tokenizers, code and analysis plan.[1]

[1]Code, results and the analysis plan: https://github.com/arjanchaudharyy/nepali-pretokenizer-repair. R2 tokenizers: https://huggingface.co/Aarjan/Llama-3.2-Nepali-Repaired-Tokenizer-32k, https://huggingface.co/Aarjan/Qwen3-Nepali-Repaired-Tokenizer-32k.

II. Related work

Tokenization premiums. Tokenizers trained mostly on English encode other languages in more tokens, so those languages pay more for the same content [8–11]. For Nepali, Kadariya [12] reports 4.79 for `cl100k`, and Srivastava [13] finds that `o200k` cuts the mean Indic premium from 8.0 to 2.1 without attributing the change to the regex. We measure sentence-level token ratios on parallel FLORES-200 text [14, 15], following Petrov et al. [9]. Fewer tokens do not guarantee a better model [16–18], so we also test the repair with continued pretraining.

Pre-tokenization of combining marks. A 2024 tiktoken issue reported that GPT-2-style regexes split Indic words at vowel signs and proposed the class `[\p{L}\p{M}]+` [19]. ZeTT [20] adjusted its regex to avoid over-segmenting Hindi and Tamil, Velayuthan and Sarveswaran [21] showed that the GPT-2, GPT-4 and Llama-3 pre-tokenizers break Tamil, Sinhala and Hindi, and Rana et al. [22] cut Hindi fertility from 3.47 to 1.36 tokens per word by swapping the GPT-2 regex for Llama-4's; see also Thakur et al. [23]. Regmi et al. [2] formalised the cause, stated the bound across 26 languages, and showed with 268M-parameter models trained from scratch that the wider class lowers Nepali bits per byte by 4.4%. Qwen3.5's released tokenizer uses a mark-aware class [24], a change also noted by Land [25], and Shrestha and Pradhan [26] study morphology-guided pre-tokenization for Nepali in models trained from scratch. The edit itself is therefore known. What has not been studied is applying it to an already trained model while keeping its English tokenization.

Adapting the vocabulary of a pretrained model. Vocabulary extension adds tokens, initialises their embeddings from existing ones [27–31] and continues pretraining [3, 32–35]; Duwal et al. [36] continue pretraining Llama-3-8B on Nepali with 4-bit QLoRA and without new tokens. Continued BPE appends new merges to a byte-level model's merge list; Purason et al. [5] introduce it and report tokenizer-only compression for many languages, Nepali included, under the unchanged regex. Yamaguchi et al. [4] and Yamaguchi et al. [37] extend Llama-3 and Qwen models for several abugidas, Smith et al. [6] expand LFM2 and Mehta et al. [38] Qwen3. Didenko [39] adapts models to Ukrainian at fixed vocabulary size, keeping English token counts within 0.05%. All of these keep the original pre-tokenizer. Tokenizer transfer [20, 40, 41], dynamic tokenization [42] and grapheme- or akshara-aware pre-tokenizers [43, 44] replace more of the tokenizer. Dagan et al. [45] replace the tokenizer of a pretrained model, regex included, during code fine-tuning and find it recovers after about 50B tokens in their setting; Rana et al. [22] likewise give Llama-3.2-1B a new tokenizer with Llama-4's mark-aware regex and train its embedding and output layers on 40B tokens. Both alter English tokenization; ours does not.

The closest prior system is TituLLM [46], which extends Llama-3.2 for Bengali; its released v2.0 tokenizer uses a `\w`-based regex that keeps vowel signs attached. It does not identify the regex as a cause and has no regex-only or vocabulary-only arm, and its regex also changes digit and whitespace pre-tokens in English. We add a controlled attribution on deployed models and a repair that leaves every input without marks or Devanagari characters unchanged.

III. Pre-tokens bound merge-based extension

A byte-level BPE tokenizer [47, 48] first splits a string s with a regex into pre-tokens $P(s)$ and then encodes each pre-token on its own by applying merges in rank order. Every pre-token yields at least one token, so $|T(s)| \geq |P(s)|$; Regmi et al. [2] formalise this as a training-free lower bound on fertility. Appending merges, whatever they are and however many, changes neither the normalizer nor the regex, so the bound holds for every tokenizer extended this way.

We turn this into a bound on the premium. For a tokenizer T with pre-tokenizer P, the *pre-token floor* is $|P(\text{ne})|/|T(\text{en})|$, the number of Nepali pre-tokens divided by the number of English tokens, summed over the 1,012 FLORES-200 devtest sentences. A merge-based extension that keeps P and leaves English unchanged cannot bring the premium below this value, though it may stop above it. Our standard extension R1 leaves English unchanged: every new merge produces a token that contains a complete Devanagari code point, and since UTF-8 is self-synchronising, no new merge can fire on text without one (the same argument appears in the proof of Proposition 1, Appendix C).

For today's regexes the floor is far from slack. In Llama-3 and Qwen the word branch is `[^\r\n\p{L}\p{N}]?\p{L}+`, a run of letters optionally preceded by one other character. A Devanagari vowel sign ends the current run and becomes the leading character of the next one, so *nepālko sarkār* ("government of Nepal") becomes six pre-tokens, four of which start with a vowel sign that belongs to the previous consonant (Figure 4, Appendix J). The floor is 2.19 for Llama-3 and 2.17 for Qwen3. That is above the premium of `o200k` (1.61), and 2.7 to 2.8 times the floor under the repaired regex of §V (0.79 and 0.80). Llama-3's premium is already 2.58, so standard extension can lower it by at most 15%.

Released extensions end at the floor. Figure 1 measures released merge-based extensions of letters-only models; Appendix I lists every number and how we found them. Ten extensions by Yamaguchi et al. of Llama-3, Llama-3.1, Qwen2.5 and Qwen3 [4, 37] cover six scripts written with combining marks. All ten end within 4% of their floor, having closed at least 98% of the distance from the base tokenizer.[2] The 61k-token in-place expansion of LFM2-8B-A1B into LFM2.5-8B-A1B [6] ends further from its floor (1.07 times it for Hindi, 1.16 for Nepali, 1.24 for Bengali and 1.82 for Thai, which is written without spaces), but its floors are just as high. Under the repaired regex the same text has 2.2 to 5.9 times fewer pre-tokens, so each of these extensions stopped well short of what new vocabulary could achieve. Yamaguchi et al.'s Amharic

[2]These extensions re-order the base merges and change English token counts slightly (+6.5% for the Llama-3 Telugu release), so for them we compare tokens with pre-tokens of the target text directly.

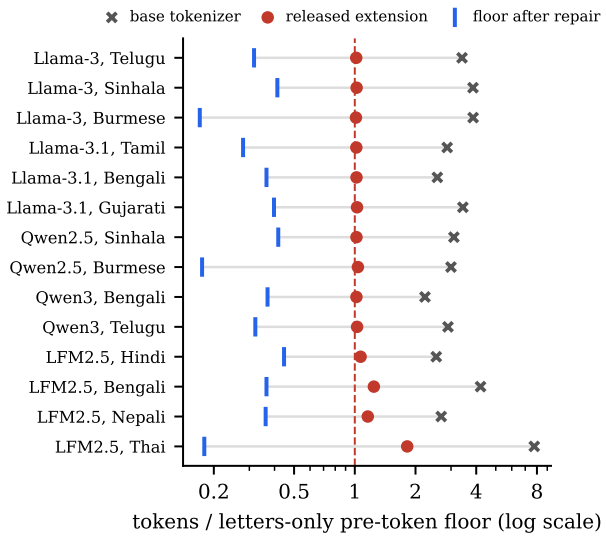


Fig. 1. Released merge-based extensions of letters-only models on FLORES-200 devtest of each target language, relative to the letters-only pre-token floor. The ten extensions by Yamaguchi et al. (top) move from the base tokenizer (×) to within 4% of the floor (●). LFM2.5 (bottom) stops further above it in all four languages. The blue tick is the floor under the repaired regex. Full numbers, including the Amharic control, are in Table VI.

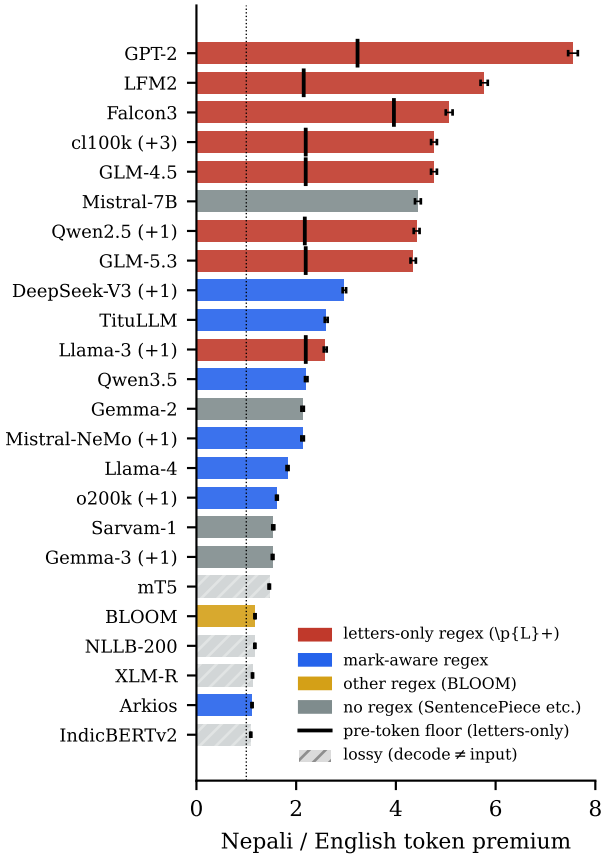


Fig. 2. Nepali/English premium of 24 distinct tokenizers (33 released; "+n" counts aliases) on FLORES-200 devtest, with 95% paired-bootstrap intervals. Black ticks mark the pre-token floor of each letters-only tokenizer: no merge-based extension can move its bar to the left of the tick. Dotted line: parity with English.

extension, for a script without combining marks, stays at 1.57 times its floor, and the repair does not change that floor.

Added tokens bypass the floor. Strings registered as Hugging Face *added tokens* are matched in the raw text before the regex runs, so the bound does not apply to them. Sinhala-Llama [49] extends Llama-3 for Sinhala this way and goes below its letters-only floor. To test this route fairly, we registered as added tokens the 31,753 whole Devanagari strings among the 32,000 learned for R2 (§V), on the unchanged regex. Nepali then costs 1.025 times English on Llama-3 and 1.009 on Qwen3, close to R2's 1.014, and no FLORES-200 sentence without a Devanagari character changes its token ids. The floor therefore binds merge-based extension only. The price is the segmentation. Added tokens match the longest string anywhere, ignoring word boundaries, and 21% of the matches start inside a word and 28% end inside one. Allowing only whole-word matches (`single_word=True`) removes those splits and also the gain: the premium returns to 2.49 on Llama-3 and 4.22 on Qwen3 (Appendix J). We did not train models with added tokens.

IV. Audit of Deployed Tokenizers

We measured 33 released tokenizers on FLORES-200 devtest. Nine produce token ids identical to another tokenizer's on every sentence of five languages we checked (for example, Phi-4, OLMo-2 and Granite-4.1 match `cl100k`, and Qwen3 matches Qwen2.5), which leaves 24 distinct tokenizers (Figure 2, Appendix F). We classify each by its own pre-tokenizer configuration, since models in one family do not always share a regex. Four SentencePiece and WordPiece tokenizers (XLM-R, mT5, NLLB-200, IndicBERTv2) do not round-trip their input, so their low premiums are not comparable; we mark them and exclude them from comparisons.

Eight distinct tokenizers (13 released models, including GLM-5.3 and LFM2) use the letters-only class, and none of them can be extended below 2.15. Leaving aside GPT-2 (7.55), which we include only for reference, their premiums range from 2.58 (Llama-3) to 5.77 (LFM2). The seven lossless tokenizers with a mark-aware regex range from 1.12 (Arkios [50]) to 2.97 (DeepSeek-V3).

We split each premium into three factors (Appendix G). The base term, 0.77 to 0.80 for every tokenizer, is what the premium would be if every Nepali pre-token under the repaired regex were a single token. The regex factor, 2.77 to 2.79 for the Llama-3, Qwen and `cl100k` pattern, counts the extra pre-tokens the tokenizer's own regex creates. The vocabulary factor, tokens per pre-token, is the only one that merge-based extension can reduce, and it cannot go below 1. For Llama-3 it is already 1.18, so most of the remaining premium sits in the regex factor. The factors are an accounting identity. The regex factor measures room for improvement, and repairing the regex alone does not lower the premium (§VII-A). Among

mark-aware tokenizers the vocabulary factor still ranges from 1.41 (Arkios) to 3.71 (DeepSeek-V3), so a good regex also needs Nepali merges. Within families, the Nepali premium halves from Qwen3 to Qwen3.5 (4.42 to 2.20), whose word branch is close to our repaired class [24], and falls from 2.58 to 1.83 from Llama-3 to Llama-4, which adopts `o200k`'s pattern. Both vocabularies also grew, so these changes are consistent with our account without isolating the regex.

A well-formed Devanagari syllable never starts with a combining mark, yet 55% of Llama-3's Nepali tokens do. Even `o200k`'s mark-aware regex leaves 26%, because BPE also splits inside words.

V. Repairing a deployed tokenizer

The repair has three parts. We edit the regex, learn new merges on top of the existing ones, and initialise the new embeddings so that the model behaves exactly as before on old tokens.

Regex edit. We add `\p{M}` to the word class and exclude it from the optional leading character and from the punctuation class. A mark can then only be matched as part of a word, so it stays with the letters around it. For the Llama-3 and Qwen pattern this takes two string substitutions that edit three character classes (Appendix B); digits, contractions, whitespace and punctuation are handled as before. We call the result the *repaired regex*. Qwen3.5 ships almost the same word branch, as its released tokenizer file shows [24].

Continued BPE. Following Purason et al. [5], we pre-tokenize Nepali text with the arm's regex, segment every pre-token with the original merges, as the deployed model sees it, and run BPE over these segmentations with each base token as an indivisible symbol. The K new merges are appended after the existing ones, so the original merges act first. We discard any merge whose output contains no complete code point in U+0900–U+097F, together with merges that depend on it. R1 and R2 run this procedure on the same text with the same K and differ only in the regex.

Proposition 1 (English invariance)**.** *Let s be a normalised string that contains no combining mark (`\p{M}`) and no code point in U+0900–U+097F. Then the repaired tokenizer* R2 *and the original tokenizer produce identical token ids for s.*

The proof is in Appendix C. The proposition covers ASCII English and code and every script whose normalised text has no combining marks. It does not cover other scripts that use marks: the general repair changes the pre-tokens of 61 of the 204 FLORES-200 languages, including Arabic, Bengali and Thai, whose tokenization the base model has seen only in its original form.

A Devanagari-only repair. Admitting only the 34 combining marks of the Devanagari block, instead of all of `\p{M}`, gives a stronger guarantee by the same argument: token ids are unchanged for every input without a code point in U+0900–U+097F. On FLORES-200 this variant changes pre-tokens only in the ten Devanagari languages, and on FLORES Nepali its R2 token ids are identical to the general repair's. On the FineWeb-2 Nepali test split (275 MB, one document per line), 24 of 49,462 lines are pre-tokenized differently, all because of marks from other blocks, such as emoji variation selectors and Kannada vowel signs (Appendix B). We recommend this variant for deployment. All experiments in this paper, and the released tokenizers, use the general repair.

Embedding initialisation. Each new token's input and output rows are set to the mean of the rows of its constituent base tokens [27], obtained by applying the original merges to its byte-level string. Original rows are untouched, so on any input whose ids are all original the resized model gives the same logits for every original token as the base model; on an English probe the largest difference is below 10^{-4}. The new tokens do add terms to the softmax denominator, and continued pretraining is needed to train them.

Loading pitfall. In transformers 5.x, the class `Qwen2Tokenizer` rebuilds the pre-tokenizer from a hard-coded pattern when it loads, which silently undoes a repaired regex stored in `tokenizer.json`. Our released tokenizers therefore set `tokenizer_class` to `PreTrainedTokenizerFast`, and we check after loading that the premium matches the paper.

Arms. R0 (no extension) keeps the original tokenizer. R1 (standard extension) appends K merges learned under the original regex, as in Purason et al. [5] and Yamaguchi et al. [4]. R2 (repair + extension) switches to the repaired regex and appends K merges learned under it.

VI. Experimental setup

We committed an analysis plan to our own repository before training any model, then seven amendments: budget cuts and the primary analysis (before any result), warm-up runs (after the first Llama results), a Llama rerun with the beginning-of-sequence (BOS) token in training, matched-steps and second-seed runs, further seeds with a learning-rate check, R0 at the higher rate, and a missing second seed (each of the last five before running it). The study up to the BOS rerun took about 11 hours, and several analysis choices changed after we saw results; Appendix A lists every commit and deviation, and Appendix D the training and evaluation details.

Models and data. We use Llama-3.2-1B (base) [51] and Qwen3-0.6B-Base [52], whose tokenizers ship the letters-only regex and which tie input and output embeddings. The Qwen3 tokenizer is identical across model sizes; we built it on Qwen3-1.7B-Base. Nepali text is FineWeb-2 `npi_Deva` [53] and English is FineWeb-Edu [54]. We normalise to NFC, deduplicate, and remove every document that shares a whitespace 10-gram with FLORES-200 dev or devtest. A 30 MB held-out slice of each language is never trained on. R1 and R2 learn merges on the same 1 GB of Nepali, with the registered K=32,000.

Continued pretraining. Each arm is trained for one epoch on the same documents, 1 GB of Nepali and 1 GB of English interleaved in one fixed order, with identical optimisation: a fixed, untuned peak learning rate of 10^{-4} and cosine decay (later amendments add second seeds and, on Qwen, a rate of 3×10^{-4}). Because R2 encodes the same bytes in fewer tokens, it takes fewer steps. R0 and R1 also save a checkpoint

at the step where R2 ends, before their learning rate has fully decayed. We call these the *main runs*. The *warm-up runs* (Llama only) repeat them after first training only the embedding matrix on 10% of the stream at a learning rate of 10^{-3}.

Evaluation. All metrics are normalised by UTF-8 bytes. The primary metric is bits per byte (BPB) on held-out native Nepali, *byte-matched*: each document is split at whitespace into pieces of about 2,000 bytes, and every arm scores the same pieces. Each piece is scored after one context token. The *registered prefix* is the BOS token for Llama and the end-of-text token for Qwen, whose tokenizer defines no BOS token. In the main and warm-up runs, however, documents were separated only by the end-of-text token, so the Llama models never saw BOS. We therefore also score them after that token, the *training separator*, as a diagnostic chosen after seeing results, and we reran the Llama main runs with BOS at every document start (§VII-B). Secondary metrics (held-out English BPB, FLORES BPB, Belebele, 5-shot translation) are in Appendix K.

Analysis. The registered main hypothesis was that R2 is non-inferior to both R1 and R0 on held-out Nepali BPB, with a margin of 0.01, while using fewer training tokens. The labels for the 95% interval of (R2 − other) are applied in a fixed order: *better* if it lies entirely below 0; otherwise *non-inferior* if it lies entirely below $+0.01$; otherwise *worse* if it lies entirely above 0; otherwise *no detectable difference*. So $[+0.004, +0.007]$ is non-inferior although it excludes zero. The label "worse" was added in the second amendment. The registered test was a paired bootstrap over held-out pieces. Pieces of one document are correlated, so we report a bootstrap over documents instead (B=10,000). Clustering by the 503 held-out documents (Appendix D) widens the Nepali intervals 1.4 to 2.6 times and changes no verdict. The same test on English pieces is the registered English control. All intervals describe the sampling of evaluation text. Run-to-run variance is measured only by second seeds (Appendix M).

VII. Results

A. Compression

Figure 3 sweeps the number of new merges K. Under the letters-only regex (R1), extension saturates almost at once. For Llama-3 the first 1,000 merges take the premium from 2.58 to 2.32, and the next 63,000 only to 2.20, within 0.01 of the floor. For Qwen3, R1 reaches 2.19 by K=16k and stops at 2.18, against a floor of 2.17. It stops early because the letters-only regex leaves few distinct Nepali pre-tokens to merge (Appendix J). Under the repaired regex (R2), additional merges keep helping. At K=32k Nepali reaches parity with English (1.01) on both models, and at 64k it is cheaper than English (0.95), below `o200k`. At K=32k, R2 needs 46% of R1's tokens for FLORES Nepali. All 28 extended tokenizers decode losslessly and reproduce the original token ids on all 1,012 English devtest sentences, as §III and Proposition 1 predict.

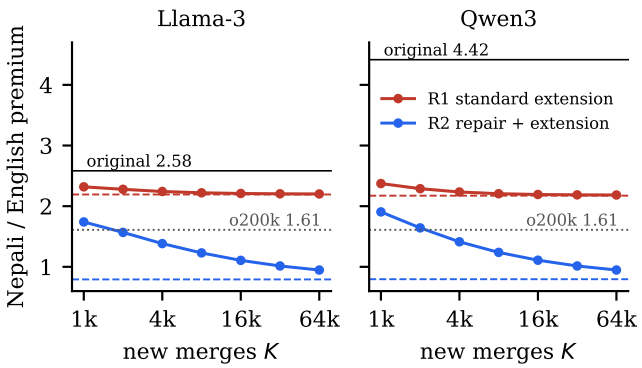


Fig. 3. Nepali/English premium on FLORES-200 devtest after adding K merges by continued BPE, under the letters-only regex (R1) and the repaired regex (R2). Dashed lines: pre-token floors under each model's own pattern (2.19 and 2.17 letters-only; 0.79 and 0.80 repaired, for Llama-3 and Qwen3). 95% paired-bootstrap intervals (half-width at most 0.03) are narrower than the markers.

Both parts of the repair are needed. Repairing the regex without new merges changes almost nothing (2.61 for Llama-3, 4.42 for Qwen3), because the old merges still cut Nepali words into the old pieces. The repair also changes where tokens start. The share of Nepali tokens that start with a combining mark, a rough sign of a split syllable, rises from 55% (Llama-3) and 32% (Qwen3) to 65% and 64% under R1, since new merges can only build on mark-initial fragments, and falls to 13% and 11% under R2. The mark-initial share does not predict model quality in §VII-B: R2 has by far the fewest mark-initial tokens yet trails R0 on both models.

We repeated the sweep for seven other languages written with combining marks and two controls (Table V, Appendix H), learning merges on 25 to 361 MB of FineWeb-2 text per language. At K=32k, R2 brings all seven to between 0.88 and 1.24 times English. R1 ends within 5% of its floor in six of them. Thai is the exception: it is written without spaces, so its letters-only pre-tokens are long and leave R1 room. For Amharic and Korean, whose scripts have no combining marks, R1 and R2 are identical at every K, as the mechanism predicts.

B. Continued pretraining

The registered outcome. Table I gives the results. The main hypothesis fails against R0: continued pretraining without new tokens gave the lowest held-out Nepali BPB on both models. On Qwen, R2 was worse than R0 by 0.070, far beyond the 0.01 margin, and R1 was worse by 0.060. Against standard extension, R2 − R1 = $+0.0094$ $[+0.0084, +0.0104]$ (registered piece-level bootstrap: $[+0.0086, +0.0101]$), labelled "worse" although the point estimate lies inside the margin; averaged over two seeds it is $+0.0122$ $[+0.0112, +0.0132]$.

The BOS-consistent rerun. The main Llama runs never saw BOS (§VI). Under the registered prefix they give R2 − R1 = $+0.0058$ $[+0.0043, +0.0073]$, non-inferior although it excludes zero, but $+0.027$ after the training separator. A

	Llama-3.2-1B				Qwen3-0.6B
	main runs		warm-up runs	BOS rerun	main runs
	registered prefix	training sep.	training sep.	registered prefix	(reg. = sep.)
Held-out Nepali BPB (lower is better)					
Base model, no CPT	0.766	n/a	n/a	0.766	0.852
R0	0.474	0.471	0.471	0.471	0.501
R1	0.534	0.483	0.480	0.483	0.561
R2	0.539	0.510	0.501	0.509	0.570
Paired difference in held-out Nepali BPB (positive: first arm worse), document-clustered 95% interval					
R1 − R0	+0.060 [+0.058, +0.061]	+0.012 [+0.011, +0.013]	+0.010 [+0.009, +0.011]	+0.012 [+0.011, +0.013]	+0.060 [+0.058, +0.062]
R2 − R0	+0.066 [+0.064, +0.068]	+0.039 [+0.038, +0.041]	+0.031 [+0.029, +0.032]	+0.038 [+0.037, +0.039]	+0.070 [+0.067, +0.072]
R2 − R1	+0.006 [+0.004, +0.007]	+0.027 [+0.026, +0.028]	+0.021 [+0.020, +0.022]	+0.026 [+0.024, +0.027]	+0.009 [+0.008, +0.010]
Training tokens (M)		417 / 386 / 286		418 / 386 / 286	574 / 392 / 292
Decoding steps		2.55 / 2.17 / 1			4.35 / 2.16 / 1

TABLE I

CONTINUED PRETRAINING, SEED 0 OF EACH ARM, ON THE SAME 1 GB OF NEPALI AND 1 GB OF ENGLISH. **OUR RESULTS ARE THE BOS-RERUN (LLAMA) AND QWEN COLUMNS**; TWO-SEED MEANS AND MATCHED-STEPS RUNS ARE IN TABLE IX; THE OTHER LLAMA COLUMNS SHOW THE SCORING ARTIFACT OF §VII-B. BPB: HELD-OUT NATIVE NEPALI, IN PIECES OF ABOUT 2,000 BYTES, SCORED AFTER THE REGISTERED PREFIX (BOS FOR LLAMA; END-OF-TEXT FOR QWEN, WHICH HAS NO BOS) OR THE TRAINING SEPARATOR (END-OF-TEXT; A POST HOC DIAGNOSTIC). INTERVALS RESAMPLE 503 HELD-OUT DOCUMENTS AND REFLECT ONLY THE SAMPLING OF EVALUATION TEXT. TRAINING TOKENS AND DECODING STEPS (FLORES NEPALI, RELATIVE TO R2) ARE R0 / R1 / R2. SECONDARY METRICS: TABLE VIII.

third amendment retrained the three Llama arms with BOS before every document and nothing else changed; we take this rerun as our Llama result. R0, R1 and R2 reach 0.471, 0.483 and 0.509: R1 is worse than R0 by 0.0124 $[0.0114, 0.0135]$ and R2 worse than R1 by 0.0256 $[0.0245, 0.0267]$, about 5% (0.0252 over two seeds). These match the separator diagnostic within 0.002, so the main runs' registered-prefix numbers reflect a scoring artifact of the unseen BOS token; after the end-of-text token the rerun scores 0.875 to 1.666 (Appendix K).

Matched steps. At equal bytes, R2 takes 26% fewer steps than R1. A fourth amendment retrained R2 for R1's step count (737 for Llama with BOS, 748 for Qwen), drawing the extra 35% of windows from a second pass over its stream, and a fifth added second seeds. Averaged over two seeds, matched-steps R2 is *better* than R1 on Qwen by 0.0099 $[0.0088, 0.0109]$ (0.549 against 0.559) and worse on Llama by 0.0120 $[0.0111, 0.0129]$ (0.495 against 0.483); R0 stays best, by 0.049 and 0.024. Matching steps thus reverses the equal-bytes gap on Qwen and halves it on Llama, though with 35% of the data repeated. Seeds move each arm by about 0.003 or less (Appendix M).

Learning rate. A fifth and a sixth amendment repeated the Qwen arms at 3×10^{-4} instead of 10^{-4} (one seed, R2 at matched steps). Every arm improves, R0 by 0.021, R1 by 0.052 and R2 by 0.046, but the order holds: R0 0.479, R2 0.504, R1 0.509. R2 beats R1 by 0.0057 $[0.0047, 0.0066]$ and trails R0 by 0.024 $[0.023, 0.026]$, so our predictions that R0 would gain little and R2 would match it failed. English gets worse: 0.875 to 0.879 BPB, against 0.862 to 0.864 at 10^{-4} and 0.873 for the base model.

What the repair buys. The clear gain is in token count. For the same FLORES Nepali text, R2 needs 46% of R1's tokens on both models, and 39% (Llama) and 23% (Qwen) of R0's. We have not shown faster generation of equally good text: our timings forced 128 generated tokens per model on shared GPUs, used the original Llama runs and compared different outputs. In them R2's generations carry 1.55 (Llama) and 1.47 (Qwen) times as many bytes per token as R1's, and throughput is 1.48 and 1.50 times higher, which we treat as indicative only.

English. The registered English control holds: R2 − R0 on held-out English is -0.0010 $[-0.0014, -0.0007]$ (Llama rerun) and -0.0028 $[-0.0032, -0.0024]$ (Qwen), and R2 beats R1 by 0.0022 on both. "English-preserving" refers to token ids only: every Llama arm ends worse on English than the base model (0.798 to 0.804 against 0.788).

VIII. DISCUSSION

Recommendations. Before extending a letters-only model for a language with combining marks, compare the target text's pre-tokens with its tokens; if they are close, as for Llama-3 on Nepali, merges have little room. Under our recipes, plain continued pretraining gave the lowest held-out Nepali BPB, as Zhao et al. [55] found for Chinese, so extension needs another reason, such as token cost. Then the repair is worth considering: at matched steps it was within about 0.012 BPB of standard extension either way and needs 54% fewer tokens for the same Nepali text, though we have not shown a latency gain for equally good outputs. We would use the Devanagari-only repair (untrained here; identical to ours on FLORES Nepali), load it with a class that does not rebuild its pre-tokenizer, and score models after the context they were trained with.

Why our sign differs from Regmi et al. From scratch the wider class lowers Nepali BPB by 4.4% at equal compute and still leads at equal bytes [2]; retrofitted at equal bytes it is worse, and the gap decreases under longer training with repeated data [cf. 26]: matched steps flip the sign on Qwen and halve the gap on Llama.

Limitations

- **The main hypothesis failed.** At our budget, neither extension recipe beat continued pretraining without new tokens on Nepali BPB. At equal bytes the repair was also worse than standard extension on both models; at matched steps it was better on Qwen and still worse on Llama.
- **Scale.** We adapt 1B- and 0.6B-parameter models with 1 GB of text per language and one epoch; 1 GB of Nepali also holds less content than 1 GB of English [56]. The registered 3 GB runs were dropped for budget reasons, so we cannot say whether the gap between arms closes with more data; Dagan et al. [45] found recovery from a tokenizer change only after about 50B tokens in their code-adaptation setting.
- **Matched steps repeat data.** The matched-steps R2 runs reach R1's step count by seeing 35% of their stream twice; they hold compute equal, not data, though repeating data for a few epochs costs little [57]. We did not run R0 at matched steps, and Nepali is still a smaller share of R2's training tokens (26% against 45% for R1).
- **Scoring two tokenizations.** For the seed-0 equal-bytes models we add the base tokenization to the canonical one, a partial step towards the marginal likelihood [58, 59]. It changes no score, because the models give the base tokenization almost no probability; this is a weak check, since nearer alternatives, such as splitting one new token, could still carry mass.
- **Few runs.** Every interval reflects only the sampling of evaluation text. In the Llama BOS rerun and on Qwen, every arm at 10^{-4} has two seeds, which differ by about 0.003 or less per arm; the runs at 3×10^{-4} have one.
- **Clustering.** Nepali intervals resample the 503 held-out documents; English intervals still cluster pieces by a length heuristic.
- **Learning rate.** We tried two learning rates, on Qwen only and with one seed at the higher one; continued-pretraining schedules matter [60], and the English loss at the higher rate suggests forgetting. Moving from 10^{-4} to 3×10^{-4} improved every arm by 0.02 to 0.05 without changing their order, so the best rate for each arm may differ from both and Llama was not tuned at all. Mean initialisation may also suit R2's long new tokens less well than R1's short ones [61].
- **BOS in training.** The main and warm-up Llama runs never saw BOS, which the registered evaluation puts first, so their registered scores and all their secondary metrics carry this artifact; the warm-up translation scores are unusable. The rerun with BOS in training resolves the main comparison, but only on BPB: we did not run Belebele, translation or generation for it, nor a warm-up version. Qwen has no BOS token and was not affected.
- **Weak secondary metrics.** Belebele accuracy is near chance for every model, translation scores come from a single greedy pass without intervals and keep only the first output line, which is empty whenever a model begins its answer with a newline (we did not count how often), and measured generation speed was taken on shared GPUs (Appendix K).
- **Equal compute.** The equal-compute checkpoints of R0 and R1 are taken before their learning rate has fully decayed, and are scored on 1 MB only.
- **Languages.** We train models for Nepali only. For other languages we report compression, and the general repair, which we trained and released, changes the tokenization of 61 FLORES-200 languages whose quality we did not measure, including Hindi and Thai, which Llama-3.2 officially supports. The cross-script tokenizers use a relaxed merge filter not covered by Proposition 1 (Appendix H).
- **Added tokens and segmentation.** We measured the added-token route around the floor only at the tokenizer level and trained no model with it; Churchill and Skiena [7] add tokens post hoc to Llama 3.2 1B for whole characters, not words, and Balde et al. [62] change how added tokens are matched. Our objection to it is segmentation, and we have no evidence that segmentation matters for quality. Whether the regex repair beats added tokens is untested.
- **Scoring context.** The BOS-consistent rerun scores every piece after BOS, but 1,756 of the 2,259 pieces start mid-document, a context these models never saw after BOS in training.
- **What would settle it.** A learning-rate sweep on both models, better embedding initialisation and a curve over data budgets would show whether R0's lead and the Llama gap persist.

Ethics and use of AI assistance

All text and evaluation data are public (FineWeb-2, FineWeb-Edu, FLORES-200, Belebele) and used under their licences; we collected no data from people. Released Llama-derived tokenizers follow the Llama 3.2 Community License [63]; Qwen3-derived tokenizers are Apache-2.0.

We used a large language model assistant extensively. Under our direction it wrote and ran the experiment code, searched the literature and drafted the text. We reviewed and approved the study design, the paper and all citations, and take responsibility for the content. The tokenizer-side numbers can be reproduced from public tokenizers and data with our released code, and every continued-pretraining statistic from the released per-piece scores.

References

[1] OpenAI, "GPT-2 source code: `src/encoder.py`," GitHub repository openai/gpt-2, 2019. [Online]. Available: https://github.com/openai/gpt-2/blob/master/src/encoder.py

[2] S. Regmi, S. Pudasaini, and C. P. Pun, "Vowel signs are not letters: A pre-tokenization ceiling on multilingual tokenizer fertility," 2026. [Online]. Available: https://arxiv.org/abs/2608.26449

[3] A. Tejaswi, N. Gupta, and E. Choi, "Exploring design choices for building language-specific LLMs," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 10 485–10 500. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.614/

[4] A. Yamaguchi, A. Villavicencio, and N. Aletras, "How can we effectively expand the vocabulary of LLMs with 0.01GB of target language text?" *Computational Linguistics*, vol. 52, no. 1, pp. 295–330, 2026.

[5] T. Purason, P. Chizhov, I. P. Yamshchikov, and M. Fishel, "Teaching old tokenizers new words: Efficient tokenizer adaptation for pretrained models," in *Findings of the Association for Computational Linguistics: EACL 2026*, V. Demberg, K. Inui, and L. Marquez, Eds. Rabat, Morocco: Association for Computational Linguistics, Mar. 2026, pp. 6492–6516. [Online]. Available: https://aclanthology.org/2026.findings-eacl.341/

[6] J. T. H. Smith, T. Dakhran, A. Cabrera, S. S. Lee, P. Pak, A. Tadimeti, T. Seyde, M. Labonne, A. Amini, and M. Lechner, "In-place tokenizer expansion for pretrained LLMs," 2026.

[7] G. Churchill and S. Skiena, "Reducing tokenization premiums for low-resource languages," 2026. [Online]. Available: https://arxiv.org/abs/2601.13328

[8] P. Rust, J. Pfeiffer, I. Vulić, S. Ruder, and I. Gurevych, "How good is your tokenizer? on the monolingual performance of multilingual language models," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, C. Zong, F. Xia, W. Li, and R. Navigli, Eds. Online: Association for Computational Linguistics, Aug. 2021, pp. 3118–3135. [Online]. Available: https://aclanthology.org/2021.acl-long.243/

[9] A. Petrov, E. La Malfa, P. H. S. Torr, and A. Bibi, "Language model tokenizers introduce unfairness between languages," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/74bb24dca8334adce292883b4b651eda-Abstract-Conference.html

[10] O. Ahia, S. Kumar, H. Gonen, J. Kasai, D. Mortensen, N. Smith, and Y. Tsvetkov, "Do all languages cost the same? Tokenization in the era of commercial language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 9904–9923. [Online]. Available: https://aclanthology.org/2023.emnlp-main.614/

[11] J. M. Lundin, A. Zhang, N. Karim, H. Louzan, G. Wei, D. I. Adelani, and C. Carroll, "The token tax: Systematic bias in multilingual tokenization," in *Proceedings of the 7th Workshop on African Natural Language Processing (AfricaNLP 2026)*, E. A. Chimoto, C. Lignos, S. Muhammad, I. Abdulmumin, C. Siro, and D. I. Adelani, Eds. Rabat, Morocco: Association for Computational Linguistics, Mar. 2026, pp. 103–112. [Online]. Available: https://aclanthology.org/2026.africanlp-main.10/

[12] R. Kadariya, "How well does AI understand Nepali? A multi-task benchmark of large language models and embedding models on a low-resource language," Preprint (Zenodo/ResearchGate), Jul. 2026. [Online]. Available: https://zenodo.org/records/21320729

[13] P. Srivastava, "The tokenizer tax: Quantifying and explaining the cross-lingual cost of subword tokenization for Indian languages," 2026. [Online]. Available: https://arxiv.org/abs/2607.24276

[14] N. Goyal, C. Gao, V. Chaudhary, P.-J. Chen, G. Wenzek, D. Ju, S. Krishnan, M. Ranzato, F. Guzmán, and A. Fan, "The Flores-101 evaluation benchmark for low-resource and multilingual machine translation," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 522–538, 2022. [Online]. Available: https://aclanthology.org/2022.tacl-1.30/

[15] NLLB Team, M. R. Costa-jussà, J. Cross, O. Çelebi, M. Elbayad, K. Heafield *et al.*, "No language left behind: Scaling human-centered machine translation," *arXiv preprint arXiv:2207.04672*, 2022. [Online]. Available: https://arxiv.org/abs/2207.04672

[16] C. W. Schmidt, V. Reddy, H. Zhang, A. Alameddine, O. Uzan, Y. Pinter, and C. Tanner, "Tokenization is more than compression," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 678–702. [Online]. Available: https://aclanthology.org/2024.emnlp-main.40/

[17] V. Zouhar, C. Meister, J. Gastaldi, L. Du, M. Sachan, and R. Cotterell, "Tokenization and the noiseless channel," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 5184–5207. [Online]. Available: https://aclanthology.org/2023.acl-l

ong.284/

[18] N. Luitel, N. Bekoju, A. K. Sah, and S. Shakya, "Can perplexity predict fine-tuning performance? an investigation of tokenization effects on sequential language models for Nepali," 2025. [Online]. Available: https://arxiv.org/abs/2404.18071

[19] ajaykg, "Combining marks and Indic vowel marks within words are being split breaking all Indic languages and most languages except English and CJKs (tiktoken issue #292)," GitHub issue, openai/tiktoken, 2024. [Online]. Available: https://github.com/openai/tiktoken/issues/292

[20] B. Minixhofer, E. M. Ponti, and I. Vulić, "Zero-shot tokenizer transfer," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024, pp. 46 791–46 818.

[21] M. Velayuthan and K. Sarveswaran, "Egalitarian language representation in language models: It all begins with tokenizers," in *Proceedings of the 31st International Conference on Computational Linguistics*, O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. D. Eugenio, and S. Schockaert, Eds. Abu Dhabi, UAE: Association for Computational Linguistics, Jan. 2025, pp. 5987–5996. [Online]. Available: https://aclanthology.org/2025.coling-main.400/

[22] S. Rana, A. Kulkarni, A. Menezes, C. Khatri, and S. Agarwal, "MUTANT: A recipe for multilingual tokenizer design," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. San Diego, California, United States: Association for Computational Linguistics, Jul. 2026, pp. 46 261–46 277. [Online]. Available: https://aclanthology.org/2026.acl-long.2146/

[23] A. Thakur, A. Nagpal, A. Savarkar, K. Pundalik, S. Dosi, P. Sawarkar, V. Thakur, R. Saluja, M. S. Desarkar, and G. Ramakrishnan, "The art of breaking words: Rethinking multilingual tokenizer design," 2025. [Online]. Available: https://arxiv.org/abs/2508.06533

[24] Qwen Team, "Qwen3.5-9B tokenizer configuration (`tokenizer.json`)," https://huggingface.co/Qwen/Qwen3.5-9B, 2026, pre-tokenizer pattern read from the released file.

[25] S. Land, "Small pre-tokenization bugs with a big multilingual price," Substack blog post, Sep. 2026. [Online]. Available: https://tokencontributions.substack.com/p/small-pre-tokenization-bugs-with

[26] K. Shrestha and N. Pradhan, "Preserving morphemes: Morphology-guided pre-tokenization for Nepali," 2026. [Online]. Available: https://arxiv.org/abs/2609.33395

[27] L. Gee, A. Zugarini, L. Rigutini, and P. Torroni, "Fast vocabulary transfer for language model compression," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*, Y. Li and A. Lazaridou, Eds. Abu Dhabi, UAE: Association for Computational Linguistics, Dec. 2022, pp. 409–416. [Online]. Available: https://aclanthology.org/2022.emnlp-industry.41/

[28] B. Minixhofer, F. Paischer, and N. Rekabsaz, "WECHSEL: Effective initialization of subword embeddings for cross-lingual transfer of monolingual language models," in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, M. Carpuat, M.-C. de Marneffe, and I. V. Meza Ruiz, Eds. Seattle, United States: Association for Computational Linguistics, Jul. 2022, pp. 3992–4006. [Online]. Available: https://aclanthology.org/2022.naacl-main.293/

[29] K. Dobler and G. de Melo, "FOCUS: Effective embedding initialization for monolingual specialization of multilingual models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 13 440–13 454. [Online]. Available: https://aclanthology.org/2023.emnlp-main.829/

[30] Y. Liu, P. Lin, M. Wang, and H. Schuetze, "OFA: A framework of initializing unseen subword embeddings for efficient large-scale multilingual continued pretraining," in *Findings of the Association for Computational Linguistics: NAACL 2024*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 1067–1097. [Online]. Available: https://aclanthology.org/2024.findings-naacl.68/

[31] F. Remy, P. Delobelle, H. Avetisyan, A. Khabibullina, M. de Lhoneux, and T. Demeester, "Trans-tokenization and cross-lingual vocabulary transfers: Language adaptation of LLMs for low-resource NLP," in *First Conference on Language Modeling (COLM)*, 2024. [Online]. Available: https://arxiv.org/abs/2408.04303

[32] Y. Cui, Z. Yang, and X. Yao, "Efficient and effective text encoding for Chinese LLaMA and Alpaca," 2023. [Online]. Available: https://arxiv.org/abs/2304.08177

[33] S. Kim, S. Choi, and M. Jeong, "Efficient and effective vocabulary expansion towards multilingual large language models," 2024.

[34] Z. Csaki, B. Li, J. L. Li, Q. Xu, P. Pawakapan, L. Zhang, Y. Du, H. Zhao, C. Hu, and U. Thakker, "SambaLingo: Teaching large language models new languages," in *Proceedings of the Fourth Workshop on Multilingual Representation Learning (MRL 2024)*, J. Sälevä and A. Owodunni, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 1–21. [Online]. Available: https://aclanthology.org/2024.mrl-1.1/

[35] A. Nag, S. Chakrabarti, A. Mukherjee, and N. Ganguly, "Efficient continual pre-training of LLMs for low-resource languages," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*. Albuquerque, New Mexico: Association for

Computational Linguistics, Apr. 2025, pp. 304–317. [Online]. Available: https://aclanthology.org/2025.naacl-industry.25/

[36] S. Duwal, S. Prasai, and S. Manandhar, "Domain-adaptative continual learning for low-resource tasks: Evaluation on Nepali," in *Proceedings of the First Workshop on Challenges in Processing South Asian Languages (CHiPSAL 2025)*. Abu Dhabi, UAE: International Committee on Computational Linguistics, Jan. 2025, pp. 144–153. [Online]. Available: https://aclanthology.org/2025.chipsal-1.14/

[37] A. Yamaguchi, T. Morishita, A. Villavicencio, and N. Aletras, "Adapting chat language models using only target unlabeled language data," *Transactions on Machine Learning Research*, 2025. [Online]. Available: https://mlanthology.org/tmlr/2025/yamaguchi2025tmlr-adapting/

[38] M. Mehta, N. Subramani, Z. Xu, A. Gupta, and V. Srikumar, "Defragmenting language models: An interpretability-based approach for vocabulary expansion," 2026. [Online]. Available: https://arxiv.org/abs/2604.16656

[39] B. Didenko, "Writing-system-level tokenizer adaptation for byte-level BPE," 2026, tokShop @ COLM 2026, non-archival. [Online]. Available: https://arxiv.org/abs/2608.00582

[40] B. Minixhofer, I. Vulić, and E. M. Ponti, "Universal cross-tokenizer distillation via approximate likelihood matching," in *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, 2025, pp. 88 034–88 063.

[41] K. Dobler and G. de Melo, "Language adaptation on a tight academic compute budget: Tokenizer swapping works and pure bfloat16 is enough," 2024, wANT@ICML 2024.

[42] D. Feher, I. Vulić, and B. Minixhofer, "Retrofitting large language models with dynamic tokenization," 2024.

[43] S. H. Kapila and R. Bagavathy, "Type-driven tokenization for Brahmic scripts," 2026. [Online]. Available: https://arxiv.org/abs/2609.22125

[44] K. Darshana, "Separate before you compress: The WWHO tokenization architecture," 2026. [Online]. Available: https://arxiv.org/abs/2603.25309

[45] G. Dagan, G. Synnaeve, and B. Rozière, "Getting the most out of your tokenizer for pre-training and domain adaptation," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 9784–9805. [Online]. Available: https://proceedings.mlr.press/v235/dagan24a.html

[46] S. K. Nahin, R. N. Nandi, S. Sarker, Q. S. Muhtaseem, M. Kowsher, A. C. Shill, M. Ibrahim, M. H. Menon, T. A. Muntasir, and F. Alam, "TituLLMs: A family of Bangla LLMs with comprehensive benchmarking," in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 24 922–24 940. [Online]. Available: https://aclanthology.org/2025.findings-acl.1279/

[47] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, K. Erk and N. A. Smith, Eds. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 1715–1725. [Online]. Available: https://aclanthology.org/P16-1162/

[48] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[49] H. W. K. Aravinda, R. Sirajudeen, S. Karunathilake, N. de Silva, R. Kaur, and S. Ranathunga, "SinLlama – a large language model for Sinhala," in *2025 Moratuwa Engineering Research Conference (MERCon)*. IEEE, 2025, pp. 617–622.

[50] S. Regmi, S. Pudasaini, and C. P. Pun, "Arkios: An open bilingual English-Nepali language model trained from scratch, with a Devanagari-aware tokenizer," 2026. [Online]. Available: https://arxiv.org/abs/2608.30092

[51] A. Grattafiori *et al.*, "The Llama 3 herd of models," 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[52] A. Yang *et al.*, "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[53] G. Penedo, H. Kydlíček, V. Sabolčec, B. Messmer, N. Foroutan, A. H. Kargaran, C. Raffel, M. Jaggi, L. Von Werra, and T. Wolf, "FineWeb2: One pipeline to scale them all – adapting pre-training data processing to every language," in *Second Conference on Language Modeling (COLM)*, 2025. [Online]. Available: https://arxiv.org/abs/2506.20920

[54] G. Penedo, H. Kydlíček, L. Ben Allal, A. Lozhkov, M. Mitchell, C. Raffel, L. Von Werra, and T. Wolf, "The FineWeb datasets: Decanting the web for the finest text data at scale," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024. [Online]. Available: https://arxiv.org/abs/2406.17557

[55] J. Zhao, Z. Zhang, L. Gao, Q. Zhang, T. Gui, and X. Huang, "LLaMA beyond English: An empirical study on language capability transfer," 2024. [Online]. Available: https://arxiv.org/abs/2401.01055

[56] C. Arnett, T. A. Chang, and B. Bergen, "A bit of a problem: Measurement disparities in dataset sizes across languages," in *Proceedings of the 3rd Annual Meeting of the Special Interest Group on Under-resourced Languages @ LREC-COLING 2024*. Torino, Italia: ELRA and ICCL, May 2024, pp. 1–9. [Online]. Available: https://aclanthology.org/2024.sigul-1.1/

[57] N. Muennighoff, A. M. Rush, B. Barak, T. Le Scao,

A. Piktus, N. Tazi, S. Pyysalo, T. Wolf, and C. Raffel, "Scaling data-constrained language models," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2305.16264

[58] K. Cao and L. Rimell, "You should evaluate your language model on marginal likelihood over tokenisations," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021, pp. 2104–2114. [Online]. Available: https://aclanthology.org/2021.emnlp-main.161/

[59] N. Chirkova, G. Kruszewski, J. Rozen, and M. Dymetman, "Should you marginalize over possible tokenizations?" in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, 2023, pp. 1–12. [Online]. Available: https://aclanthology.org/2023.acl-short.1/

[60] A. Ibrahim, B. Thérien, K. Gupta, M. L. Richter, Q. Anthony, T. Lesort, E. Belilovsky, and I. Rish, "Simple and scalable strategies to continually pre-train large language models," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=DimPeeCxKO

[61] R. Joshi, U. Vaidya, S. S. Chauhan, and N. Wartikar, "Beyond initialization loss: A systematic study of token embedding initialization strategies for LLM vocabulary extension," 2026.

[62] G. Balde, S. Roy, M. Mondal, and N. Ganguly, "Adaptive BPE tokenization for enhanced vocabulary adaptation in finetuning pretrained language models," in *Findings of the Association for Computational Linguistics: EMNLP 2024*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 14 724–14 733. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.863/

[63] Meta Platforms, Inc., "Llama 3.2 Community License Agreement," https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/LICENSE, 2024, version release date 25 Sep 2024; accessed 2026-10-04.

[64] L. Bandarkar, D. Liang, B. Muller, M. Artetxe, S. N. Shukla, D. Husa, N. Goyal, A. Krishnan, L. Zettlemoyer, and M. Khabsa, "The Belebele benchmark: a parallel reading comprehension dataset in 122 language variants," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 749–775. [Online]. Available: https://aclanthology.org/2024.acl-long.44/

[65] M. Choudhury, S. Chauhan, R. J. Das, D. Sahnan, X. Han, H. Li, A. Singh, A. A. Jadhav, U. Agarwal, M. Choudhary, D. Banerjee, F. Koto, J. Bhat, A. Shukla, S. Ghosh, S. Kamboj, O. Pandit, L. Pradhan, R. Pal, S. Sahu, S. Doraiswamy, P. Mullah, A. El Filali, N. Sengupta, G. Ramakrishnan, R. Joshi, G. Gosal, A. Sheinin, N. Vassilieva, and P. Nakov, "Llama-3-Nanda-10B-Chat: An open generative large language model for Hindi," 2025.

APPENDIX A
PRE-REGISTRATION HISTORY

We committed the analysis plan to the project repository in nine steps between 2026-10-04 and 2026-10-06 (IST), and report all nine. Times are commit times.

1) **Original registration** (`7713fd2`, 17:59) registered hypotheses about the mechanism, a regex-only tokenizer ablation, a cross-script dose–response and training from scratch. Our literature check then found that Regmi et al. [2] had published the mechanism, the ablation, the cross-script analysis and a from-scratch comparison five weeks earlier. We withdrew those hypotheses as contributions.
2) **Retrofit registration** (`5430e86`, 18:16), before any model was trained or evaluated, registered the study reported here: arms R0, R1, R2 and the untrained base model; $K{=}32{,}000$ unless R2 saturated earlier on FLORES dev; equal-bytes continued pretraining of Llama-3.2-1B and Qwen3-1.7B-Base on a fixed byte budget split equally between Nepali and English (3 GB each in our plan, a figure first written down in the first amendment); a learning rate selected once per model on R0; two seeds. Its hypotheses were P1 (compression ordering, with R1 above the letters-only floor), P2, the main hypothesis (held-out Nepali BPB of R2 at most 0.01 above both R1 and R0, while using fewer training tokens), P3 (English BPB of R2 and R0 within 0.01) and P4 (R2 decodes Nepali faster at equal bytes). P1 had already been measured when it was written, so we report it as a measurement.
3) **First amendment** (`aa8779a`, 19:50; file `PREREG_AMENDMENT.md`, which numbers the amendments from 2 because it counts the retrofit registration as the first), written after an internal review and before any continued-pretraining run had produced an evaluation, recorded the deviations below, fixed four pipeline bugs, and fixed the primary analysis: byte-matched BPB, a paired bootstrap over pieces and three claim labels (better, non-inferior, no detectable difference; §VI gives the later ordered set). Earlier launches had been stopped by us because of the bugs, and none produced an evaluated checkpoint. The only earlier evaluation, of the untouched base Llama-3.2-1B with a since-fixed prefix bug, is discarded.
4) **Second amendment** (`9be7667`, 21:46; `PREREG_AMENDMENT3.md`) was written *after* observing the Llama R0 and R2 main-run results and before any R1 or Qwen result. It registered the warm-up runs, with the prediction that they shrink the R2 − R0 gap, and 3 GB runs that we did not run, and it added the "worse" category to the claim rules.
5) **Third amendment** (`660fd50`, 2026-10-05, 03:24; `PREREG_AMENDMENT4.md`) was written after all main and

warm-up results and before running what it registered: the three Llama main runs repeated with every training document wrapped as BOS, document, end-of-text, scored with the registered protocol. It predicted that scores after BOS and after the end-of-text token would agree within 0.005 for every arm, and that R2 − R1 after BOS would lie between the two earlier values ($+0.006$ and $+0.027$). The second prediction held ($+0.026$), and the rerun matches the earlier separator diagnostic within 0.002, which confirms that diagnostic in spirit. The first prediction failed: after the end-of-text token the rerun models score far worse (Appendix K).

6) **Fourth amendment** (`9fb12e7`, 2026-10-05, 16:08; `PREREG_AMENDMENT5.md`) was written after all earlier results and two external reviews, and before running what it registered: R2 retrained for R1's step count on Llama (BOS setting) and Qwen, Qwen R1 and R2 with a second seed, and two-tokenization scoring. It predicted that matched-steps R2 would be non-inferior to R1 on Qwen (it was better), smaller-gapped but still worse on Llama (held), still worse than R0 on both (held), that a second Qwen seed would move R2 − R1 by less than 0.005 (it moved by 0.0057, so this failed narrowly), and that two-tokenization scoring would change no verdict (held).
7) **Fifth amendment** (`c8bc301`, 2026-10-06, 00:48; `PREREG_AMENDMENT6.md`), written after the fourth amendment's results and before running, added second seeds for matched-steps R2 (both models), Llama R0 and R1 and Qwen R0, and Qwen R1 and matched-steps R2 at a learning rate of 3×10^{-4}, under a budget cap. It predicted that with seeds averaged matched-steps R2 would be better than R1 on Qwen and worse on Llama, that R0 would stay best, and that R2 − R1 would keep its sign at 3×10^{-4}. All four held.
8) **Sixth amendment** (`4bc56cc`, 2026-10-06, 05:01; `PREREG_AMENDMENT7.md`), written after the fifth amendment's results and before running, added Qwen R0 at 3×10^{-4}. It predicted that R0 would gain less than 0.005 and that matched-steps R2 would be non-inferior to it. Both failed: R0 gained 0.021 and stays ahead by 0.024.
9) **Seventh amendment** (`6dddbeb`, 2026-10-06, 07:30; `PREREG_AMENDMENT8.md`), written after an external review and before running, added a second seed for equal-bytes Llama R2. It predicted that the two-seed R2 − R1 would stay worse and within 0.005 of $+0.0256$; it held ($+0.0252$ $[+0.0241, +0.0263]$).

Two departures from these amendments: the fourth registered two-tokenization scoring also for its new runs, but we applied it only to the six seed-0 equal-bytes models of the Llama rerun and Qwen; and the first amendment stated that we would not tune the learning rate on results, whereas the fifth and sixth test a second rate after seeing results. We use that rate only as a robustness check, not to select one.

In the repository these are `PREREG.md`, `PREREG_RETROFIT.md`, `PREREG_AMENDMENT.md`, and `PREREG_AMENDMENT3.md` to `PREREG_AMENDMENT8.md`. The first four commits lie within four hours, the fifth about six hours later, and the last three over the following day. All are in a repository we control, so they document our plan without an external timestamp. Apart from the author line of `PREREG.md`, no plan file was edited after its commit.

Outcome of each registered hypothesis. P1 holds as a measurement (§VII-A). P2 fails against R0 on both models. Against R1 it fails for Llama in the BOS-consistent rerun (R2 is worse by 0.026), and it also fails for Qwen (R2 is worse by 0.009, 0.012 over two seeds); it held for Llama only in the main runs under the registered prefix, a result of the BOS artifact (§VII-B). P3 holds. P4, faster generation in bytes per second at equal generated bytes, was not reliably evaluated: our timings ran on shared GPUs with a fixed number of generated tokens (Appendix K). We report token counts as a separate result. The warm-up prediction holds after the training separator; under the registered prefix the warm-up scores are dominated by the BOS artifact.

Deviations from the retrofit registration. The first amendment recorded these before any result:

- Qwen3-0.6B-Base replaces Qwen3-1.7B-Base, for budget and time. Their tokenizers are identical.
- 1 GB of Nepali and 1 GB of English per arm replace 3 GB each, with identical documents in every arm.
- A fixed learning rate of 10^{-4} replaces selection on R0 from $\{3\times10^{-5}, 10^{-4}, 3\times10^{-4}\}$ by a 200-step pilot, which was never completed (the candidate set and pilot were first recorded in the first amendment).
- One seed per arm replaces two.
- Added: R0 and R1 checkpoints at R2's final step (equal-compute view); byte-matched BPB as the primary metric, with the registered sliding-window BPB as secondary; the paired bootstrap with claim rules as the primary test; a translation cap of 512 instead of 256 new tokens.

Besides the runs added by the second to seventh amendments, we made four further changes after seeing results, all reported as such: intervals from a bootstrap over reconstructed documents instead of pieces (§VI); the training-separator diagnostic; decoding steps instead of measured bytes per second for P4, whose implementation had fixed the number of generated tokens rather than bytes; and the "worse" label from the second amendment for the main runs.

Pipeline bugs fixed before any reported run. (1) Documents had been stored one per line with their internal newlines, so "documents" were paragraphs; they are now stored as one JSON-encoded document per line. (2) New-token embeddings had been initialised by decoding tokens to text, which turns partial UTF-8 tokens into U+FFFD; constituents now come from applying the original merges to the byte-level string. (3) The initialisation check had covered Qwen's padding rows; it now covers only the original vocabulary. (4) Evaluation had prefixed documents with the end-of-text token, while base Llama expects BOS; the registered evaluation uses BOS where it exists.

APPENDIX B
REPAIR DETAILS

Regex edit. For the Llama-3 and Qwen pattern we make two string substitutions. The first replaces `\p{L}+` by `[\p{L}\p{M}]+` in the word branch. The second inserts `\p{M}` after `\p{L}` in the two negated classes, the optional leading character of the word branch and the punctuation branch. Three character classes change:

```
[^\r\n\p{L}\p{N}]?\p{L}+
                ↓
[^\r\n\p{L}\p{M}\p{N}]?[\p{L}\p{M}]+
␣?[^\s\p{L}\p{N}]+ → ␣?[^\s\p{L}\p{M}\p{N}]+
```

Nothing else in the pattern changes.

Devanagari-only variant. The variant replaces `\p{M}` in the three classes by an explicit class of the 34 combining marks of the Devanagari block (U+0900–0903, U+093A–093C, U+093E–094F, U+0951–0957, U+0962–0963). We checked that this class equals the set of `\p{M}` code points in the block under Python's `unicodedata` (Unicode 15.0), the `regex` module and the Hugging Face `tokenizers` engine. Every other character has the same class membership as in the original regex, so any text without a Devanagari mark is pre-tokenized exactly as before. Across the 204 FLORES-200 devtest files, the general repair changes pre-tokens in 61 languages for both models and the Devanagari-only variant in exactly the ten Devanagari languages. On FLORES Nepali the two variants give identical pre-tokens and identical R2 token ids. On the FineWeb-2 Nepali test split (275 MB, 49,462 documents, one per line), 24 lines differ, in 47 segments; the marks involved are emoji variation selectors, Kannada, Bengali, Thai and Hebrew signs and one combining candrabindu in romanised text, and the R2 token count of those lines is the same under both.

Continued BPE. At inference BPE applies the lowest-ranked applicable merge first, so the original merges act before the new ones. The extended tokenizer reproduces the training procedure except in two cases: a new merge whose output string is already in the original vocabulary can let an original merge fire again afterwards, and under Hugging Face's `ignore_merges` option (set for Llama-3) a pre-token that is itself a vocabulary entry is emitted directly. Neither case affects Proposition 1. We run merge learning inside the Hugging Face Rust trainer by mapping each base token to a private-use code point; learning $K{=}32{,}000$ merges on 1 GB of Nepali takes 1.7 to 3.4 minutes per tokenizer.

Scope of Proposition 1. The proposition covers ASCII English and code and every script whose normalised text has no combining marks, including most Cyrillic, Han, Hangul and Ethiopic text. It does not cover Devanagari, which changes by design, or text with marks, such as Hindi, Arabic with vowel points, Thai, or Latin written in decomposed form (NFD). The Llama-3 tokenizer has no normalizer, so deployments should apply NFC first; Qwen3's tokenizer applies NFC itself. It also assumes that the regex engine's `\p{M}` table matches the one used to define s; for the Devanagari-only variant we checked this directly. Empirically, R1 and R2 at $K{=}32{,}000$ give token ids identical to the original tokenizer on all 1,012 FLORES devtest sentences in English, Russian and Amharic, for both models, including one Russian sentence with a combining acute that lies outside the proposition.

APPENDIX C
PROOF OF PROPOSITION 1

Proof. Pre-tokens. The repaired regex differs from the original only in the three classes of Appendix B, and each differs from its original only on `\p{M}` characters. A regex match on s consults only characters of s, and the order of the alternatives is unchanged. Since s has no `\p{M}` character, both regexes produce the same pre-tokens. *Merges and vocabulary.* R2 differs from the original tokenizer only by appended merges and the vocabulary entries they create, each of which contains a complete code point in U+0900–U+097F. A merge (a, b) applies only where a and b are adjacent symbols of a pre-token, so its output is a contiguous substring of that pre-token's bytes. A vocabulary entry is emitted directly under `ignore_merges` only if it equals the whole pre-token. Neither can happen for a pre-token of s, because UTF-8 is self-synchronising and s contains no such code point. Each pre-token of s is therefore encoded with original merges and original vocabulary entries only, exactly as by the original tokenizer. *Ids.* New ids are appended above the original ids, so all ids coincide. □

APPENDIX D
TRAINING AND EVALUATION PROTOCOL

Data. We normalise to NFC, drop documents under 200 characters, remove exact duplicates, and drop every document that shares a whitespace 10-gram with FLORES-200 dev or devtest in Nepali or English. This also covers the Belebele passages, which are FLORES passages [64], but not its questions or answers. Documents are stored one JSON-encoded document per line. A 30 MB held-out slice of each language is carved from the cleaned stream before training and never trained on. The 1 GB of Nepali used to learn merges is taken from the start of the training stream.

Continued pretraining. Documents are interleaved and separated by the model's end-of-text token; in the main and warm-up runs no BOS token is inserted, and in the Llama BOS-consistent rerun every document is wrapped as BOS, document, end-of-text. Optimisation is identical across arms: sequence length 2,048, global batch 256 sequences (0.52M tokens), AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1), gradient clipping at 1.0, 1% linear warm-up, then cosine decay to 10% of peak, bf16 autocast with fp32 master weights, on 4×H200. Initialisation is deterministic and there is no dropout, so a second seed would change only the order of training windows and bf16 non-determinism. In the warm-up runs, stage 1 trains the whole tied embedding matrix, old rows included, at 10^{-3} on the first 10% of the stream; stage 2 is a main run started from the stage-1 model.

Evaluation. Every scored sequence is preceded by one context token (§VI), identically for every arm of a model.

- **BPB** on the held-out native slices (the first 4 MB of each) and on FLORES-200 devtest. In the primary byte-matched version, each held-out document is split at whitespace into pieces of at most 2,000 bytes and every piece is scored independently, so every arm conditions on the same bytes. Token windows would give R2 more bytes of context per scored token. As a secondary metric, whole documents are scored with a sliding window of 2,048 tokens and stride 1,024.
- **Belebele** [64] `npi_Deva` and `eng_Latn`, zero-shot, scoring each answer string by its log-likelihood per byte given passage and question.
- **Translation**: FLORES-200 devtest Nepali↔English, 5-shot from FLORES dev, greedy decoding capped at 512 new tokens, first output line, chrF++.
- **Generation speed**: UTF-8 bytes generated per second in greedy continuation of FLORES dev Nepali sentences (batch 64, 128 new tokens).

Document ids were not stored with the pieces. The Nepali held-out slice holds 503 documents (the first 503 of FineWeb-2 `npi_Deva` after cleaning; `results/revision/heldout_ne_doc_ids.json`), recovered after an external review, and all Nepali intervals resample these documents. An earlier version of this paper used a length heuristic (a piece shorter than 1,900 bytes ends a document, 484 clusters); switching moved interval endpoints by at most 0.0003. English intervals still use that heuristic. The margin of 0.01 is 1.3% of the base Llama model's Nepali BPB and 1.2% of base Qwen's. The equal-compute checkpoints are scored on BPB only, on the first 1 MB of each held-out slice (567 Nepali pieces from 114 documents).

Appendix E
Data

Table II gives the corpus cleaning and the token count of each arm's training stream.

FineWeb-2 is already deduplicated and filtered for length, which is why our own filters remove almost nothing.

Appendix F
Full audit

Table III lists every distinct tokenizer. For each of the nine aliases we compared token ids with its partner on all 1,012 devtest sentences of English, Nepali, Hindi, Chinese and Arabic; all were identical. Kimi-K3 (requires executing remote code) and Mistral-Large-3 (its transformers load does not round-trip; its `tokenizer.json` uses Mistral-NeMo's mark-aware regex) are not included. The four tokenizers that do not round-trip belong to encoders and translation models; we keep them as reference points.

Broken syllables. The share of Nepali tokens that begin with a combining mark is 55% for Llama-3, 32% for Qwen3 and 30% for `cl100k`. Mark-aware tokenizers show similar rates (26% for `o200k`, 55% for TituLLM, which keeps Llama-3's merges), because BPE also splits within words.

	Nepali	English
documents read	170,790	215,961
dropped: under 200 chars	0	0
dropped: exact duplicate	0	69
dropped: FLORES 10-gram	0	16
training documents	166,300	209,713
held-out (MB)	30	30
Training tokens per arm (millions)		
Llama-3.2-1B R0	207	211
Llama-3.2-1B R1	176	211
Llama-3.2-1B R2	76	211
Qwen3-0.6B R0	359	215
Qwen3-0.6B R1	177	215
Qwen3-0.6B R2	76	215

TABLE II
CORPUS CLEANING (FINEWEB-2 `npi_Deva`; FINEWEB-EDU; DOCUMENTS PLACED IN THE 30 MB HELD-OUT SLICE ARE NOT COUNTED AS TRAINING DOCUMENTS) AND THE TOKEN COUNTS OF EACH ARM'S IDENTICAL 2 GB TRAINING STREAM. ENGLISH TOKEN COUNTS DIFFER ACROSS ARMS OF A MODEL BY LESS THAN 0.012%, FROM THE FEW ENGLISH DOCUMENTS THAT CONTAIN DEVANAGARI OR COMBINING MARKS.

Appendix G
Premium decomposition

We write the premium as

$$\frac{|T(\text{ne})|}{|T(\text{en})|} = \underbrace{\frac{|P_{\text{rep}}(\text{ne})|}{|T(\text{en})|}}_{\text{base}} \cdot \underbrace{\frac{|P(\text{ne})|}{|P_{\text{rep}}(\text{ne})|}}_{\text{regex factor}} \cdot \underbrace{\frac{|T(\text{ne})|}{|P(\text{ne})|}}_{\text{vocabulary factor}},$$

where P is the tokenizer's own pre-tokenizer and P_{rep} is the repaired Llama-3 pattern, applied to every tokenizer so that the numerator of the base term is the same for all. The base lies between 0.77 and 0.80 because English token counts barely differ. Using one pattern means that Qwen's regex factor also includes its single-digit splitting. The regex factor is 0.99–1.02 for mark-aware tokenizers, 2.77–2.79 for the Llama-3, Qwen and `cl100k` pattern, 4.06 for GPT-2, whose pattern puts every run of marks in a pre-token of its own, and 5.01 for Falcon3. Falcon3's pipeline splits digits after its byte-level step, and its byte-level alphabet writes byte `0xBE` as ¾, which Unicode classes as a number, so it cuts the vowel sign `U+093E` inside its UTF-8 encoding. All floors and factors here count pre-tokens of each tokenizer's full pre-tokenizer pipeline; counting its last regex alone, as our audit script first did, understates Falcon3's floor (3.21 instead of 3.96). The decomposition is an accounting identity and does not attribute cause. The regex factor is the room that a better regex creates, which new merges must then fill.

Table IV shows the factors. For Llama-3 the vocabulary factor is 1.18, the lowest in the audit: its 128k vocabulary already nearly covers Nepali pre-tokens. For Qwen and `cl100k` both factors matter. Qwen3.5's word branch is our repaired class with only the leading-character class left unchanged; Land [25] reported its change for Hindi.

Tokenizer	Aliases	Regex	$\|V\|$	ne/en	95% CI	Floor	Mark-init.	Lossless
IndicBERTv2	–	none	250,000	1.09	[1.08, 1.10]	–	6%	no
Arkios	–	mark-aware	65,536	1.12	[1.10, 1.13]	0.79	15%	yes
XLM-R	–	none	250,002	1.12	[1.11, 1.14]	–	7%	no
NLLB-200	–	none	256,204	1.17	[1.16, 1.18]	–	6%	no
BLOOM	–	other	250,680	1.18	[1.16, 1.19]	–	17%	yes
mT5	–	none	250,100	1.46	[1.45, 1.47]	–	11%	no
Gemma-3	Gemma-4	none	262,145	1.53	[1.51, 1.54]	–	9%	yes
Sarvam-1	–	none	68,096	1.54	[1.52, 1.56]	–	14%	yes
o200k	gpt-oss	mark-aware	200,019	1.61	[1.60, 1.63]	0.80	26%	yes
Llama-4	–	mark-aware	201,135	1.83	[1.81, 1.85]	0.80	30%	yes
Mistral-NeMo	Sarvam-M	mark-aware	131,072	2.13	[2.11, 2.15]	0.79	33%	yes
Gemma-2	–	none	256,000	2.13	[2.11, 2.15]	–	19%	yes
Qwen3.5	–	mark-aware	248,077	2.20	[2.18, 2.23]	0.80	19%	yes
Llama-3	SmolLM3	letters	128,256	2.58	[2.55, 2.61]	2.19	55%	yes
TituLLM	–	mark-aware	170,497	2.60	[2.57, 2.63]	0.79	55%	yes
DeepSeek-V3	DeepSeek-V4	mark-aware	128,815	2.97	[2.93, 3.00]	0.80	38%	yes
GLM-5.3	–	letters	154,856	4.35	[4.29, 4.40]	2.19	33%	yes
Qwen2.5	Qwen3	letters	151,665	4.42	[4.36, 4.47]	2.17	32%	yes
Mistral-7B	–	none	32,768	4.44	[4.38, 4.50]	–	29%	yes
GLM-4.5	–	letters	151,365	4.76	[4.71, 4.82]	2.19	30%	yes
cl100k	Phi-4, OLMo-2, Granite-4.1	letters	100,277	4.76	[4.71, 4.82]	2.19	30%	yes
Falcon3	–	letters	131,072	5.07	[5.00, 5.14]	3.96	28%	yes
LFM2	–	letters	64,400	5.77	[5.70, 5.84]	2.15	24%	yes
GPT-2	–	letters	50,257	7.55	[7.46, 7.65]	3.23	19%	yes

TABLE III

ALL 24 DISTINCT TOKENIZERS (33 RELEASED TOKENIZERS) ON FLORES-200 DEVTEST. ALIASES: TOKENIZERS WHOSE TOKEN IDS ARE IDENTICAL ON EVERY DEVTEST SENTENCE OF FIVE LANGUAGES (APPENDIX F). REGEX: LETTERS-ONLY, MARK-AWARE, OTHER, OR NONE (NO REGEX PRE-TOKENIZER). NE/EN: NEPALI/ENGLISH TOKEN PREMIUM WITH 95% PAIRED-BOOTSTRAP INTERVAL. FLOOR: PRE-TOKEN FLOOR UNDER THE TOKENIZER'S OWN REGEX, A LOWER BOUND ON THE PREMIUM OF ANY MERGE-BASED EXTENSION (LETTERS-ONLY AND MARK-AWARE TOKENIZERS ONLY). MARK-INIT.: SHARE OF NEPALI TOKENS WHOSE FIRST CHARACTER IS A COMBINING MARK. LOSSLESS: DECODING REPRODUCES THE INPUT ON 200 NEPALI SENTENCES. ARKIOS IS FROM REGMI ET AL. [50]. HUGGING FACE REPOSITORIES ARE LISTED IN THE RELEASED CODE.

Tokenizer	Premium	Base	Regex	Vocab.
GPT-2	7.55	0.80	4.06	2.34
LFM2	5.77	0.78	2.77	2.68
Falcon3	5.07	0.79	5.01	1.28
`cl100k` (+3)	4.76	0.79	2.77	2.17
GLM-4.5	4.76	0.79	2.77	2.17
Qwen2.5 (+1)	4.42	0.78	2.79	2.03
GLM-5.3	4.35	0.79	2.77	1.98
Llama-3 (+1)	2.58	0.79	2.77	1.18
DeepSeek-V3 (+1)	2.97	0.80	1.01	3.71
TituLLM	2.60	0.79	0.99	3.31
Qwen3.5	2.20	0.78	1.02	2.76
Mistral-NeMo (+1)	2.13	0.77	1.02	2.69
Llama-4	1.83	0.80	1.00	2.30
`o200k` (+1)	1.61	0.80	1.00	2.01
Arkios	1.12	0.79	1.01	1.41

TABLE IV

PREMIUM = BASE × REGEX FACTOR × VOCABULARY FACTOR, ON FLORES-200 DEVTEST, FOR ALL 15 LETTERS-ONLY AND MARK-AWARE TOKENIZERS IN THE AUDIT (TOP: LETTERS-ONLY; BOTTOM: MARK-AWARE; "(+n)" COUNTS ALIASES). MERGE-BASED EXTENSION CAN ONLY LOWER THE LAST COLUMN, AND NOT BELOW 1.

		Llama-3			Qwen3		
Lang.	Marks	R0	R1	R2	R0	R1	R2
Hindi	31%	2.52	2.38	1.17	4.42	2.36	1.16
Bengali	32%	5.84	2.29	1.01	5.03	2.26	1.00
Tamil	35%	7.65	2.67	1.02	6.11	2.64	1.02
Telugu	36%	8.29	2.45	1.02	6.99	2.43	1.01
Sinhala	28%	8.63	2.30	1.20	6.88	2.33	1.24
Burmese	50%	11.62	3.02	1.21	8.93	2.99	1.11
Thai†	21%	2.19	1.49	0.95	2.55	1.48	0.88
Amharic	0%	7.59	0.98	0.98	4.05	0.98	0.98
Korean	0%	1.49	0.98	0.98	1.63	0.98	0.98

TABLE V

LANGUAGE/ENGLISH PREMIUM ON FLORES-200 DEVTEST, K=32,000. MARKS: SHARE OF NFC CODE POINTS THAT ARE COMBINING MARKS; AMHARIC AND KOREAN HAVE NONE (CONTROLS). †THAI IS WRITTEN WITHOUT SPACES, SO ITS PRE-TOKENS ARE LONG AND R1 IS NOT YET AT ITS FLOOR. FOR R1, MERGE LEARNING STOPPED MORE THAN 100 MERGES SHORT OF 32,000 FOR TAMIL, TELUGU, SINHALA, BURMESE AND HINDI (LLAMA) (22,325–31,815 MERGES), BECAUSE NO FURTHER MERGE WAS AVAILABLE UNDER THE LETTERS-ONLY REGEX; KOREAN AND THAI ON LLAMA END AT MOST 7 MERGES SHORT.

APPENDIX H
CROSS-SCRIPT SWEEP

For each language we used the FineWeb-2 *test* split, normalised to NFC, with documents that share a whitespace 10-gram with that language's FLORES-200 dev or devtest removed (for Thai we also checked 50-character overlaps and found none). The merge-learning text ranges from 25 MB (Amharic) to 361 MB (Bengali), less than the 1 GB used for Nepali, so for languages where R1 ran out of merges we cannot fully separate the floor from data scarcity; Sinhala and Amharic R2 may also be limited by data.

A relaxed merge filter. The Nepali filter keeps a new merge only if its output contains a complete Devanagari character. Scripts that the base vocabularies cover poorly have characters split into byte pieces, and this filter would drop the first byte-level merge of each such character and every merge built on it (for Sinhala R2 on Llama-3, only 13,530 of 32,000 merges survived). For this sweep we therefore also keep merges whose output is a byte piece of a target-script character. Such merges can fire on other scripts that share those bytes: the lead byte 0xE0 begins every code point from U+0800 to U+0FFF, which covers all Indic scripts and Thai, and a continuation-byte suffix can end code points in other blocks. They cannot fire on ASCII text, which contains no such bytes, and we verified English token ids on all 1,012 sentences for every tokenizer. Proposition 1 does not cover these tokenizers, and we did not measure their effect on other non-ASCII scripts. Hindi shares the Nepali script, and Nepali merges also change Hindi tokenization.

Thai and Burmese. Both are written without spaces between words, so a repaired pre-token can span several words. In practice no R2 Thai- or Burmese-script token spans a space in either language. In Thai, 28% (Llama-3) and 25% (Qwen3) of R2's Thai-script tokens cover more than one dictionary word (PyThaiNLP `newmm` segmentation), fewer than under R1 (33% and 31%). In Burmese, R2's tokens average 2.2 and 1.8 syllables, against 1.3 under R1 and 5.1 syllables per space-delimited chunk.

APPENDIX I
RELEASED EXTENSIONS

We did not follow a fixed search protocol. Candidates came from the papers in §II and from manual searches of Hugging Face for Llama-3, Qwen and LFM2 derivatives in Indic and Southeast Asian languages. We kept releases whose tokenizer adds merges to a letters-only base. After inspection we excluded five releases that keep the base Llama-3 tokenizer unchanged and three that do not extend a letters-only base. The Llama-3 Hindi extension of Nanda [65] is gated and could not be measured, and we found no public tokenizer for Nag et al. [35] or Purason et al. [5]. The Yamaguchi et al. extensions reorder the base merges and are not English-invariant (English tokens rise by 6.5% for Llama-3 Telugu and 7.8% for Qwen3 Telugu), so Table VI compares tokens with pre-tokens of the target text directly. Two releases do not obey the letters-only floor: TituLLM [46] changes the regex, and SinLlama [49] registers 11,080 strings as added tokens and reaches 0.54 times its letters-only floor.

APPENDIX J
ADDITIONAL TOKENIZER ARMS

Regex alone. Repairing the regex without adding merges (R2, $K{=}0$) gives 2.61 for Llama-3 (from 2.58) and 4.42 for Qwen3. Under the repaired regex the old merges still tokenize Nepali words in old pieces; the repair only removes the bound that new merges would otherwise hit.

Base	Lang.	Tokens/floor	Gap closed	Room
Llama-3-8B	te	1.016	99.3%	3.16×
Llama-3-8B	si	1.021	99.3%	2.42×
Llama-3-8B	my	1.013	99.5%	5.87×
Llama-3.1-8B	ta	1.018	99.1%	3.58×
Llama-3.1-8B	bn	1.020	98.7%	2.74×
Llama-3.1-8B	gu	1.026	98.9%	2.52×
Qwen2.5-7B	si	1.019	99.1%	2.40×
Qwen2.5-7B	my	1.036	98.2%	5.72×
Qwen3-14B	bn	1.019	98.5%	2.71×
Qwen3-14B	te	1.027	98.6%	3.11×
LFM2 (2.5)	hi	1.068	95.5%	2.24×
LFM2 (2.5)	ne	1.160	90.5%	2.77×
LFM2 (2.5)	bn	1.243	92.4%	2.74×
LFM2 (2.5)	th	1.819	87.9%	5.57×
Llama-3.1-8B	am	1.573	93.8%	1.00×

TABLE VI
RELEASED MERGE-BASED EXTENSIONS THAT KEEP THE LETTERS-ONLY REGEX, ON FLORES-200 DEVTEST OF THE TARGET LANGUAGE. TOKENS/FLOOR: EXTENDED-TOKENIZER TOKENS DIVIDED BY PRE-TOKENS UNDER ITS OWN REGEX (AT LEAST 1 BY CONSTRUCTION). GAP CLOSED: SHARE OF THE BASE-TO-FLOOR GAP THAT THE EXTENSION REMOVED. ROOM: PRE-TOKENS UNDER THE LETTERS-ONLY REGEX DIVIDED BY PRE-TOKENS UNDER THE REPAIRED REGEX. FIRST BLOCK: YAMAGUCHI ET AL. [4]; SECOND: YAMAGUCHI ET AL. [37], WITH THE TWO QWEN3-14B ROWS FROM A LATER RELEASE BY THE SAME AUTHORS; THIRD: LFM2.5 [6]; LAST: THE AMHARIC RELEASE OF YAMAGUCHI ET AL. [37], A SCRIPT WITH NO COMBINING MARKS (CONTROL).

Added tokens. Table VII gives the added-token arm of §III. Default matching reaches almost R2's compression and the lowest mark-initial rate, but splits many words at arbitrary points: a fifth of the added-token matches start inside a word and more than a quarter end inside one. With whole-word matching, few strings ever match and the premium returns close to the original.

	Premium		Mark-initial	
Tokenizer	Llama-3	Qwen3	Llama-3	Qwen3
R0	2.583	4.415	55%	32%
R1, $K{=}32$k	2.204	2.187	65%	64%
R2, $K{=}32$k	1.014	1.014	13%	11%
R2 strings, added	1.025	1.009	7%	7%
whole words only	2.486	4.219	54%	32%
Added-token share	77%	83%		
Starts mid-word	21%	21%		
Ends mid-word	28%	28%		

TABLE VII
ADDED TOKENS ON THE UNCHANGED LETTERS-ONLY REGEX, FLORES-200 DEVTEST. WE REGISTER THE 31,753 OF R2'S 32,000 NEW STRINGS THAT DECODE TO WHOLE DEVANAGARI TEXT (WITH AT MOST ONE LEADING SPACE) AS ADDED TOKENS. PREMIUM: NEPALI TOKENS / ENGLISH TOKENS. MARK-INITIAL: SHARE OF NEPALI TOKENS THAT START WITH A COMBINING MARK. THE BOTTOM ROWS DESCRIBE DEFAULT MATCHING, WHICH IGNORES WORD BOUNDARIES: THE SHARE OF TOKENS THAT ARE ADDED TOKENS, AND THE SHARE OF ADDED-TOKEN MATCHES THAT START OR END INSIDE A WORD. NO SENTENCE WITHOUT A DEVANAGARI CHARACTER CHANGES ITS TOKEN IDS (191,460 FLORES-200 SENTENCES, BOTH MODELS, BOTH MATCHING MODES).

Broken syllables. A token that starts with a combining mark has certainly split a syllable, but the converse does not hold: a token can also end inside an akshara, for example just before a virama, so these shares are a lower bound on split syllables. Counting only tokens that contain Devanagari, the share that begin with a combining mark rises from 57% (Llama-3) and 33% (Qwen3) to 67% for both under R1, and falls to 15% and 12% under R2. The letters-only regex leaves few distinct Nepali pre-tokens, so 5k to 30k merges cover nearly all frequent ones, which is why R1 saturates early.

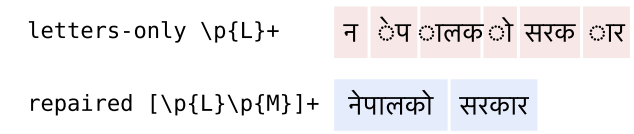


Fig. 4. Pre-tokens of *nepālko sarkār* ("government of Nepal") under the letters-only regex of Llama-3 and Qwen and under the repaired regex. We draw a dotted circle where a pre-token starts with a vowel sign; it is not a duplicated character.

APPENDIX K
ALL CONTINUED-PRETRAINING RUNS

Equal compute and training time. At R2's final step count, the R0 checkpoint of the main runs, whose learning rate had not yet decayed, is already better than the final R2 model by 0.060 $[0.056, 0.063]$ on Llama and 0.053 $[0.049, 0.057]$ on Qwen. The R1 checkpoint at that step is better by 0.006 $[0.003, 0.008]$ on Llama and level on Qwen (0.0000 $[-0.0021, +0.0020]$; final R1 is better than final R2 by 0.009 on the same pieces, so on Qwen all of R1's advantage arrives in its last 192 steps, over which its learning rate falls the last 16% of the way to its floor). These are scored on the first 1 MB of held-out text with the registered prefix, so the Llama values carry the BOS artifact of §VII-B; no such checkpoints were scored for the rerun. Measured training time fell by only 3% (Llama) and 11% (Qwen) relative to R1, in runs that shared GPUs with evaluation jobs.

Warm-up runs. Training only the embedding matrix first leaves R0 where it was (0.471 after the training separator) and improves R2 from 0.510 to 0.501. R2's deficit relative to R0 shrinks from 0.039 to 0.031, as the second amendment predicted, and its gap to R1 from 0.027 to 0.021 $[0.020, 0.022]$. These are differences between single runs, so we treat them as suggestive. The amendment stated its prediction for the registered prefix, where the warm-up makes Llama scores unusable: R0 moves from 0.474 to 0.779. With tied embeddings, the BOS row is updated as an output class even though it never appears as input, and in the warm-up runs its norm grows from 0.863 to 0.896; this may contribute.

English. In the main Llama runs under the registered prefix, R2 − R0 on English is $+0.0003$ $[-0.0002, +0.0008]$ (non-inferior). All three main-run arms are slightly worse on English than the base model (0.799 to 0.803 against 0.788).

Other BPB views. The registered sliding-window BPB, which scores whole documents, orders the Nepali arms the same way (Llama rerun 0.424, 0.437, 0.464; Qwen 0.459, 0.515, 0.525). In the BOS-consistent rerun, FLORES Nepali BPB is 0.607, 0.617 and 0.626 for R0, R1 and R2; the rerun's three held-out differences match the separator diagnostic of the main runs ($+0.0121$, $+0.0394$, $+0.0273$), and R2 − R1 lies between the two earlier values, as the third amendment predicted. At 10^{-4}, every Qwen arm ends better on English than its base model (0.862 to 0.864 against 0.873).

A failed prediction of the third amendment. The third amendment also predicted that scores after BOS and after the end-of-text token would agree within 0.005. This failed: after the end-of-text token the rerun models score 1.017, 0.875 and 1.666. Score a model after the context it was trained with.

English gains. R2 is better than R1 on English by 0.0022 on both models although English tokens are identical; a likely reason is that English is 74% of R2's training tokens but 55% of R1's, so it weighs more in R2's loss.

BOS diagnostic. On 40 held-out Nepali documents (first 1,500 characters), Llama BPB after BOS and after the end-of-text token are 0.720 and 1.865 for the base model, 0.441 and 0.438 for R0 of the main runs, and 0.758 and 0.438 for R0 of the warm-up runs. The norm of the BOS embedding is 0.863, 0.860 and 0.896 respectively (mean row norm 0.98–0.99). The base model, pretrained with BOS at sequence starts, scores far worse without it, while the continued-pretraining models, trained on fixed-length windows that start at arbitrary tokens, do not need it. The BOS-consistent rerun shows the converse: trained with BOS before every document, its models score far worse after the end-of-text token (Table VIII, sep. column). We did not test the mechanism further.

Secondary metrics. Belebele is near chance for every model. Translation scores are erratic: Qwen R2 scores 3.9 chrF++ from English to Nepali, below the untrained base model (6.8). We take the first output line as the hypothesis, which is empty when a model starts with a newline, and we did not measure how often that happens. Qwen's FLORES BPB is almost the same for R1 and R2 (0.689 and 0.688), although R2 is worse on held-out native text. We draw no conclusions from these metrics.

Measured generation speed. Generation was timed once per model, on GPUs that were also running training or evaluation jobs. The Qwen base model and Qwen R0 share tokenizer and architecture, yet differ by 41% in bytes per second, and the generated text has fewer bytes per token than FLORES (8.1 against 12.3 for Llama R2), so the measured speed also depends on what each model chose to write. With these caveats, R2 generates 1.48 (Llama) and 1.50 (Qwen) times as many bytes per second as R1, and its generations carry 8.07 and 7.47 bytes per token against 5.21 and 5.07 for R1. Section VII-B reports these together with the deterministic ratio of decoding steps on reference text.

Model	Runs	Arm	Tokens	held-out ne BPB↓ reg.	held-out ne BPB↓ sep.	en BPB↓ reg.	FLORES ne BPB	Belebele ne acc.	chrF++ ne→en	chrF++ en→ne	Gen. B/s
Llama-3.2-1B	none	base	0	0.766	n/a	0.788	0.863	0.277	32.4	10.8	5327
	main	R0	417M	0.474	0.471	0.799	0.611	0.287	32.3	19.8	5147
		R1	386M	0.534	0.483	0.803	0.648	0.268	27.2	10.4	6354
		R2	286M	0.539	0.510	0.799	0.671	0.266	27.8	12.3	9398
	warm-up	R0	417M	0.779	0.471	0.850	0.864	0.279	8.8	9.1	5349
		R1	386M	0.808	0.480	0.854	0.904	0.271	9.9	4.2	7521
		R2	286M	0.527	0.501	0.840	0.652	0.283	16.2	15.4	11078
	1 GB, BOS	R0	418M	0.471	1.017	0.799	0.607	–	–	–	–
	in training	R1	386M	0.483	0.875	0.800	0.617	–	–	–	–
		R2	286M	0.509	1.666	0.798	0.626	–	–	–	–
Qwen3-0.6B	none	base	0	0.852	0.852	0.873	0.945	0.290	26.0	6.8	2186
	main	R0	574M	0.501	0.501	0.864	0.631	0.270	20.1	13.6	3089
		R1	392M	0.561	0.561	0.864	0.689	0.253	11.6	6.9	4972
		R2	292M	0.570	0.570	0.862	0.688	0.266	17.0	3.9	7454

TABLE VIII

MAIN, WARM-UP AND BOS RERUN (SEED 0, LEARNING RATE 10^{-4}), WITH SECONDARY METRICS; SECOND SEEDS, MATCHED-STEPS AND LEARNING-RATE RUNS ARE IN TABLE IX. MAIN RUNS: 1 GB OF NEPALI AND 1 GB OF ENGLISH, DOCUMENTS SEPARATED BY THE END-OF-TEXT TOKEN ONLY. WARM-UP RUNS: THE SAME, AFTER TRAINING ONLY THE EMBEDDING MATRIX ON THE FIRST 10% OF THE STREAM (TOKENS EXCLUDES THAT STAGE). HELD-OUT BPB IS SCORED IN PIECES OF ABOUT 2,000 BYTES, AFTER THE REGISTERED PREFIX (REG.: BEGINNING-OF-SEQUENCE TOKEN FOR LLAMA) OR THE TRAINING SEPARATOR (SEP.: END-OF-TEXT TOKEN); FOR QWEN THE TWO COINCIDE. THE BASE LLAMA MODEL WAS NEVER TRAINED WITHOUT ITS BEGINNING-OF-SEQUENCE TOKEN, SO ITS SEPARATOR SCORE IS NOT MEANINGFUL (N/A). ALL COLUMNS OTHER THAN SEP. USE THE REGISTERED PREFIX; FOR THE LLAMA WARM-UP RUNS THEY ARE DOMINATED BY THE SCORING ARTIFACT OF §VII-B AND SHOULD NOT BE COMPARED WITH THE MAIN RUNS. 1 GB, BOS IN TRAINING: THE BOS-CONSISTENT RERUN, I.E. THE MAIN RUNS REPEATED WITH EVERY TRAINING DOCUMENT WRAPPED AS BOS, DOCUMENT, END-OF-TEXT (THIRD AMENDMENT); ITS SEP. COLUMN IS SCORED AFTER THE END-OF-TEXT TOKEN, WHICH THESE MODELS NEVER SAW DIRECTLY BEFORE TEXT. BELEBELE, TRANSLATION AND GENERATION SPEED WERE NOT RUN FOR IT (–). BELEBELE: ZERO-SHOT ACCURACY ON 900 ITEMS (CHANCE 0.25); EVERY VALUE LIES BETWEEN 0.25 AND 0.29 WITH 95% INTERVALS OF ABOUT ±0.03, SO IT CANNOT SEPARATE THE ARMS. CHRF++: 5-SHOT FLORES-200 DEVTEST TRANSLATION, SINGLE GREEDY PASS, NO INTERVALS, KEEPING ONLY THE FIRST LINE OF OUTPUT (EMPTY IF THE MODEL STARTS WITH A NEWLINE), SO WE DRAW NO CONCLUSIONS FROM IT. GEN. B/S: UTF-8 BYTES OF NEPALI GENERATED PER SECOND, MEASURED ON GPUS SHARED WITH CONCURRENT TRAINING RUNS; IT IS NOISY (THE BASE AND R0 QWEN MODELS SHARE TOKENIZER AND ARCHITECTURE YET DIFFER BY 41%) AND WE DO NOT DRAW CONCLUSIONS FROM IT.

APPENDIX L
WHAT THE REPAIR DOES NOT FIX

A wider word class removes the splits that the regex forces, but BPE can still split inside a syllable: 26% of `o200k`'s Nepali tokens and 11 to 13% of R2's start with a combining mark. An akshara-aware rule [43, 44] restricted to the Devanagari block would keep English token ids by the argument of Proposition 1; testing one in a deployed model is future work. Added tokens are a second route past the floor that needs no regex change; they split more words mid-way (§III), but we have not measured their effect on model quality, and our training runs give no evidence that segmentation of this kind matters for it.

APPENDIX M
ALL RUNS OF THE LATER AMENDMENTS

Table IX lists every run of the Llama BOS setting and of Qwen. Matched-steps Llama R2 is slightly worse on English than the other Llama arms (0.804 against 0.798 to 0.800), possibly because it sees part of its stream twice.

Model	Arm	LR	Seed	Steps	ne BPB	en BPB
Llama	R0	10^{-4}	0	797	0.4709	0.7985
Llama	R0	10^{-4}	1	797	0.4709	0.7984
Llama	R1	10^{-4}	0	737	0.4833	0.7998
Llama	R1	10^{-4}	1	737	0.4832	0.7993
Llama	R2	10^{-4}	0	546	0.5089	0.7975
Llama	R2	10^{-4}	1	546	0.5080	0.7974
Llama	R2-m	10^{-4}	0	737	0.4957	0.8044
Llama	R2-m	10^{-4}	1	737	0.4948	0.8043
Qwen	R0	10^{-4}	0	1095	0.5007	0.8643
Qwen	R0	10^{-4}	1	1095	0.5003	0.8631
Qwen	R1	10^{-4}	0	748	0.5609	0.8637
Qwen	R1	10^{-4}	1	748	0.5578	0.8638
Qwen	R2	10^{-4}	0	556	0.5702	0.8615
Qwen	R2	10^{-4}	1	556	0.5729	0.8617
Qwen	R2-m	10^{-4}	0	748	0.5491	0.8628
Qwen	R2-m	10^{-4}	1	748	0.5499	0.8626
Qwen	R0	3×10^{-4}	0	1095	0.4794	0.8789
Qwen	R1	3×10^{-4}	0	748	0.5092	0.8748
Qwen	R2-m	3×10^{-4}	0	748	0.5035	0.8751

TABLE IX

EVERY CONTINUED-PRETRAINING RUN OF THE LLAMA BOS SETTING AND OF QWEN, INCLUDING THE MATCHED-STEPS, SECOND-SEED AND LEARNING-RATE RUNS OF THE FOURTH TO SEVENTH AMENDMENTS. BPB: HELD-OUT TEXT IN PIECES OF ABOUT 2,000 BYTES AFTER THE REGISTERED PREFIX. R2-M: R2 TRAINED FOR R1'S STEP COUNT, WITH 35% OF ITS STREAM SEEN TWICE. SEEDS CHANGE DATA ORDER (AND BF16 NON-DETERMINISM) ONLY.